

[Login](#)

Prompt Caching: токены LLM в 10 раз дешевле — но за счёт чего?

Medium

15 min

8.3K

Programming*, Machine learning*, Artificial Intelligence, The future is here, Natural Language Processing*

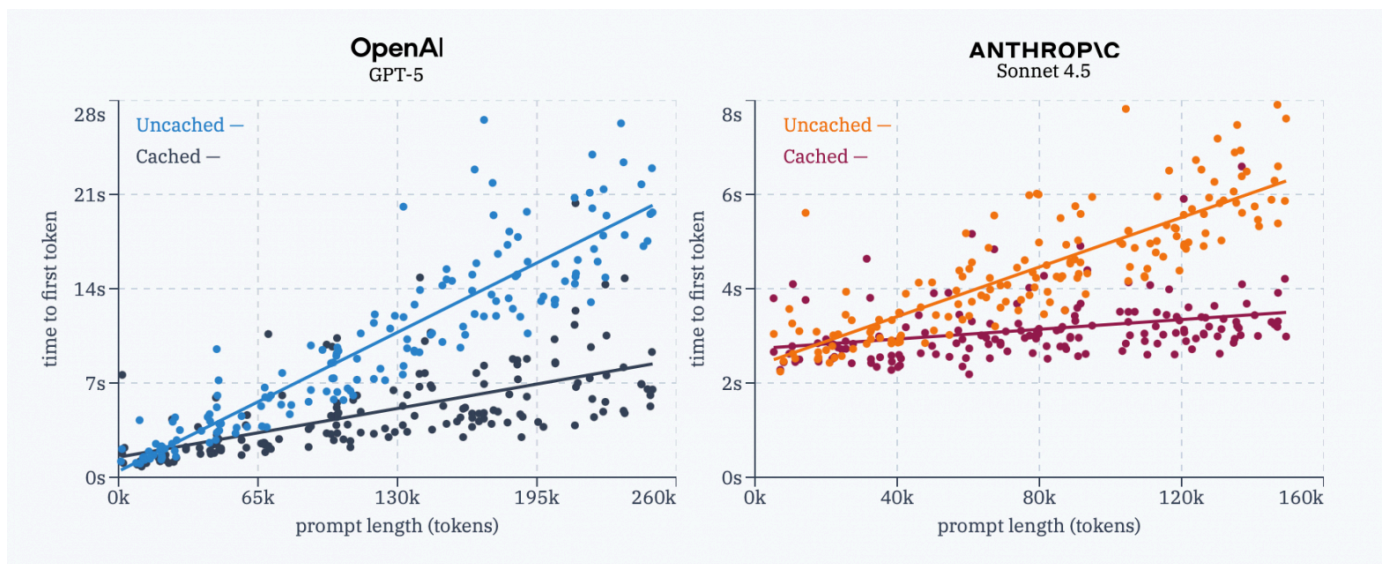
[Translation](#)

Original author: Sam Rose

Команда [AI for Devs](#) подготовила перевод и разбор статьи о Prompt Caching — технологии, которая делает входные токены LLM в разы дешевле и заметно снижает задержки. Внутри — подробное объяснение, *что именно* кэшируют OpenAI и Anthropic, как KV-кэш связан с attention в трансформерах и почему это не имеет ничего общего с повторным использованием ответов.

На момент, когда я пишу эту статью, закэшированные входные токены стоят в долларах за токен примерно в 10 раз дешевле обычных входных токенов — как в API OpenAI, так и Anthropic.

Anthropic даже заявляет, что кэширование промптов может снизить задержку «до 85% для длинных промптов», и в моих собственных тестах я убедился, что при достаточно длинном промпте это действительно так. Я отправил сотни запросов и в Anthropic, и в OpenAI и заметил существенное сокращение задержки до первого токена для промптов, в которых все входные токены брались из кэша.



Теперь, когда я зацепил вас красивым текстом и эффектными графиками, задавались ли вы когда-нибудь вопросом...

Что вообще такое закэшированный токен?

Что происходит в этих бескрайних океанах GPU, что позволяет провайдерам давать вам скидку в 10 раз на входные токены? Что именно они сохраняют между запросами? Это не сохранение ответа с последующим повторным использованием, если тот же самый промпт отправляется снова — это легко проверить через API. Напишите промпт, отправьте его десяток раз и обратите внимание: вы получаете разные ответы каждый раз, даже если в разделе usage указано, что входные токены были закэшированы.

Не удовлетворившись ответами в документации вендоров ПО для разработчиков, которые хорошо объясняют, как пользоваться кэшированием промптов, но аккуратно обходят вопрос о том, что именно кэшируется, я решил копнуть глубже. Я нырнул в кроличью нору устройства LLM, пока не понял, какие именно данные провайдеры кэшируют, для чего они используются и как это делает всё быстрее и дешевле для всех.

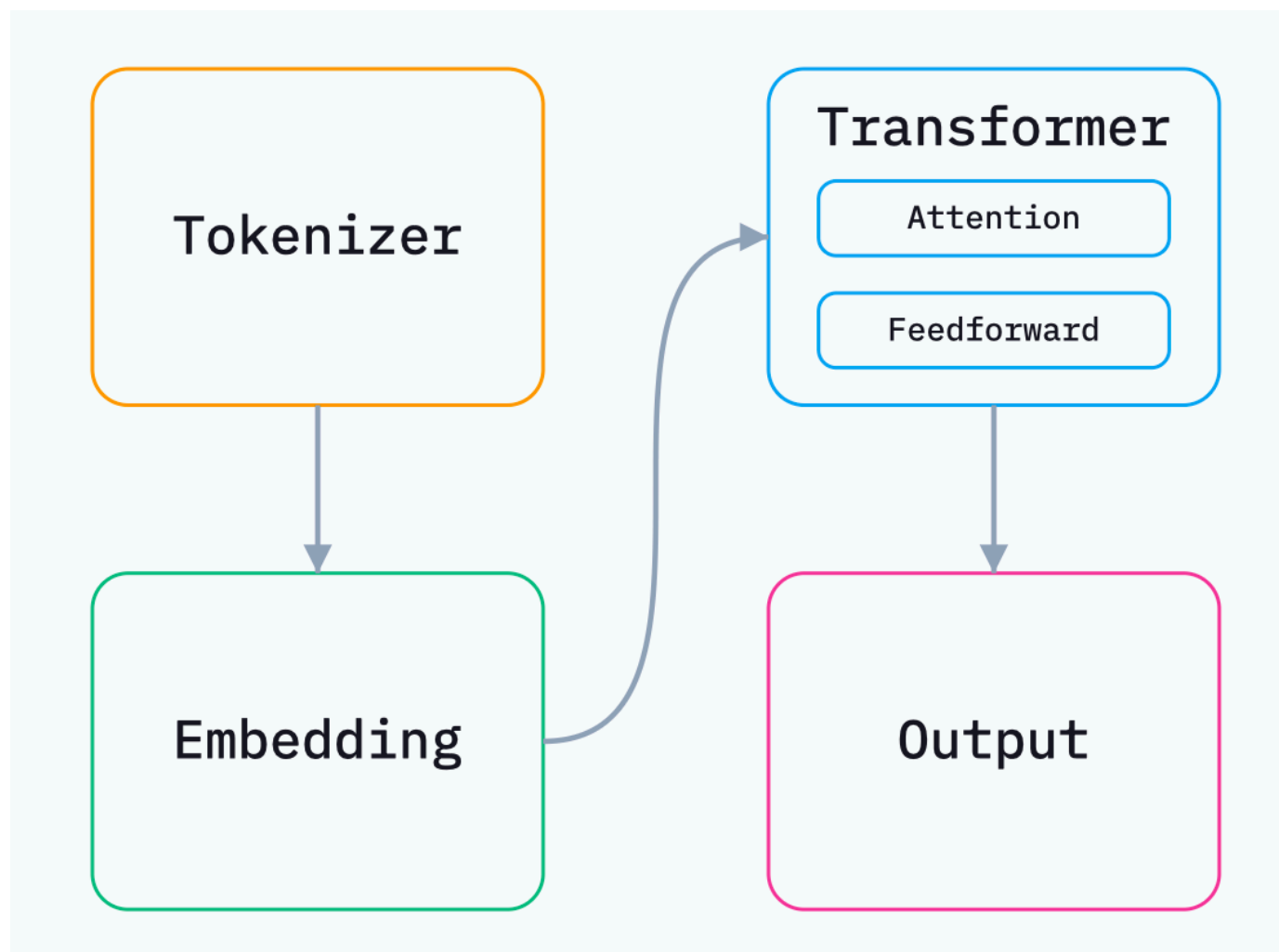
К концу этой статьи вы:

- глубже поймёте, как работают LLM
- сформируете новую интуицию о том, почему LLM устроены именно так
- разберётесь, какие именно нули и единицы кэшируются и как это снижает стоимость ваших запросов к LLM

Архитектура LLM

В своей основе LLM — это гигантские математические функции. Они принимают на вход последовательность чисел и выдают на выходе число. Внутри LLM находится огромный граф из миллиардов тщательно организованных операций, которые преобразуют входные числа в выходное.

Этот гигантский граф операций можно грубо разделить на четыре части.



Каждый узел на этой схеме можно рассматривать как функцию, которая принимает входные данные и возвращает результат. Вход подаётся в LLM в цикле до тех пор, пока специальное выходное значение не скажет модели остановиться. В виде псевдокода это может выглядеть так:

```
// Copy code
prompt = "What is the meaning of life?";

tokens = tokenizer(prompt);
while (true) {
    embeddings = embed(tokens);
    for ([attention, feedforward] of transformers) {
```

```
        embeddings = attention(embeddings);
        embeddings = feedforward(embeddings);
    }
    output_token = output(embeddings);
    if (output_token === END_TOKEN) {
        break;
    }
    tokens.push(output_token);
}

print(decode(tokens));
```

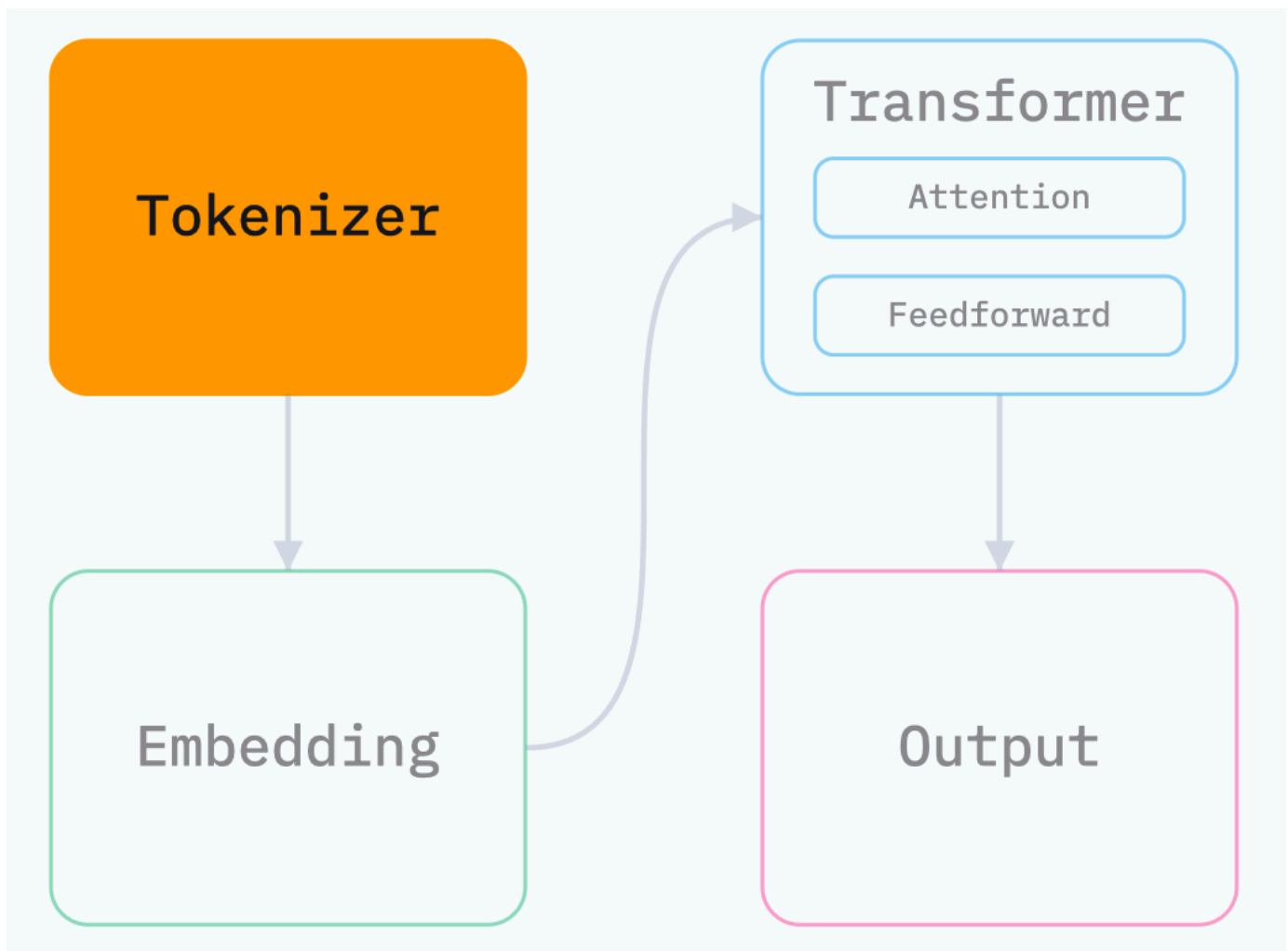
LLM удивительно компактны

Хотя пример выше сильно упрощён, меня поразило, насколько мало строк кода содержат современные LLM.

Себастьян Рашка делает самостоятельные реализации open source-моделей на PyTorch, а также выпускает массу первоклассных образовательных материалов, которые вам наверняка понравятся, если вам понравится эта статья. Например, одна из текущих ведущих открытых моделей, Olmo 3, укладывается всего в пару сотен строк кода.

Место, где происходит кэширование промптов, находится в механизме attention трансформера. Мы шаг за шагом разберём, как работает LLM, пока не доберёмся до него. А значит, начинать этот путь нужно с токенов.

Токенизатор



Прежде чем LLM сможет что-то сделать с вашим промптом, его нужно преобразовать в представление, с которым модель умеет работать. Это двухэтапный процесс, разделённый между токенизатором и стадией embedding. Зачем всё это нужно, станет понятно только когда мы дойдём до embedding, так что потерпите, пока мы разберёмся с тем, что делает токенизатор.

Токенизатор берёт ваш промпт, нарезает его на небольшие фрагменты и каждому уникальному фрагменту присваивает целочисленный идентификатор — «токен». Например, вот как GPT-5 токенизирует промпт «Check out ngrok.ai»:



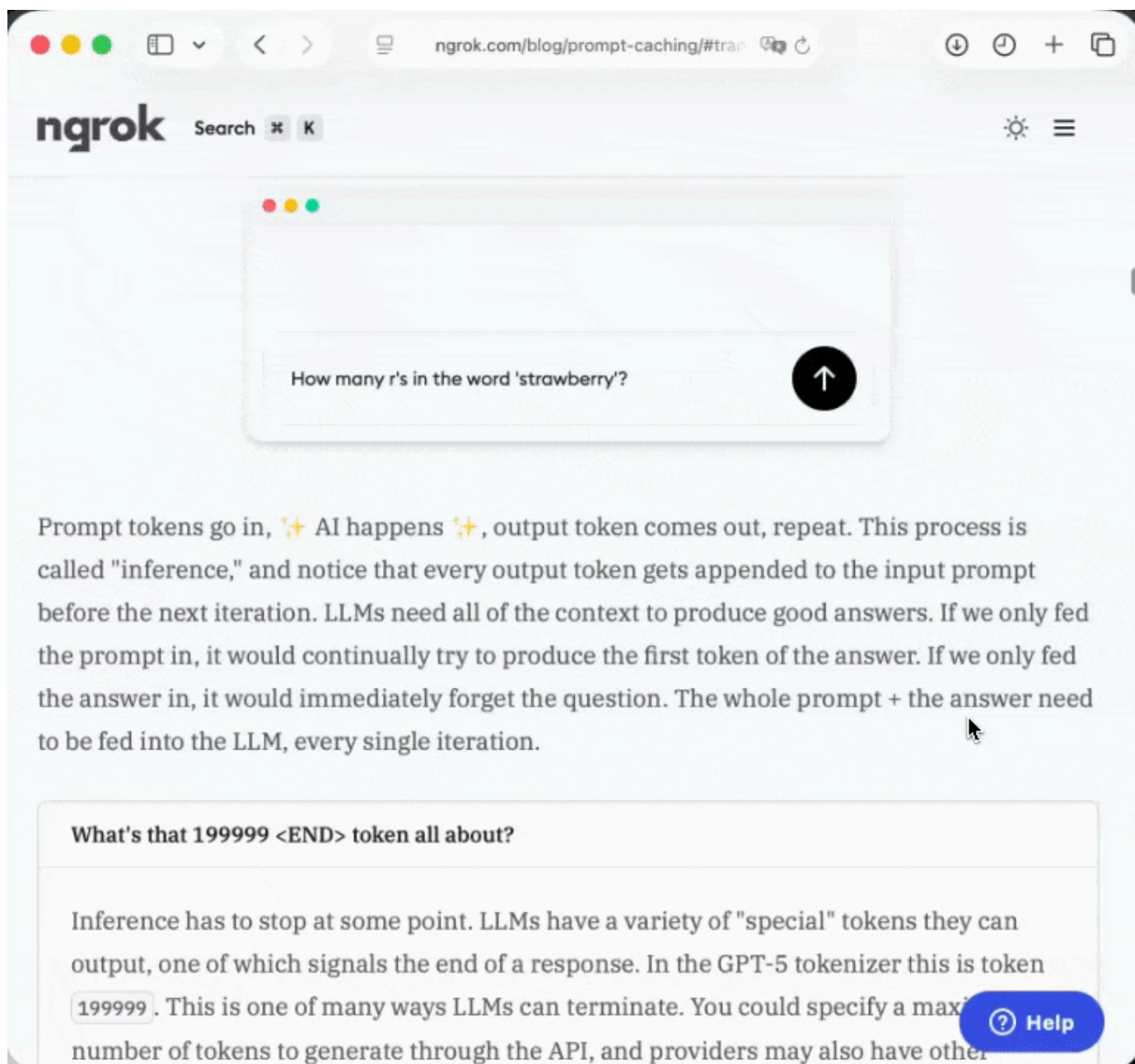
Промпт был разбит на массив ["Check", " out", " ng", "rok", ".ai"] и преобразован в токены [4383, 842, 1657, 17690, 75584] . Один и тот же промпт всегда даёт один и тот же набор токенов. Токены также чувствительны к регистру, потому что заглавные буквы несут информацию. Например, «Will» с заглавной W с большей вероятностью является именем, чем «will» со строчной.

Почему бы просто не разбивать по пробелам? Или по символам?

Это на удивление большой вопрос, и его детальное рассмотрение легко удвоило бы объём этого поста. Короткий и неудовлетворительный ответ — это компромисс. Если хочется погрузиться глубже, у Андрея Карпати есть [отличное видео](#), где он с нуля пишет токенизатор. Для кэширования промптов достаточно понимать, что токенизация превращает текст в числа.

Токены — это базовая единица ввода и вывода для LLM. Когда вы задаёте ChatGPT вопрос, ответ стримится обратно по одному токену за итерацию работы модели. Провайдеры делают это потому, что генерация полного ответа может занимать десятки секунд, а отправка токенов по мере готовности делает процесс более интерактивным.

Зададим классический вопрос к LLM, чтобы увидеть это в действии.



Токены промпта поступают на вход, ✨ происходит магия ИИ ✨, на выходе появляется токен, и всё повторяется. Этот процесс называется inference. Обратите внимание: каждый выходной токен добавляется к входному промпту перед следующей итерацией. LLM нужен весь контекст, чтобы выдавать качественные ответы. Если бы мы передавали только промпт, модель постоянно пыталась бы сгенерировать первый токен ответа. Если бы передавали только ответ — она сразу забывала бы вопрос. И промпт, и ответ целиком должны подаваться в LLM на каждой итерации.

Что это за токен 199999 ?

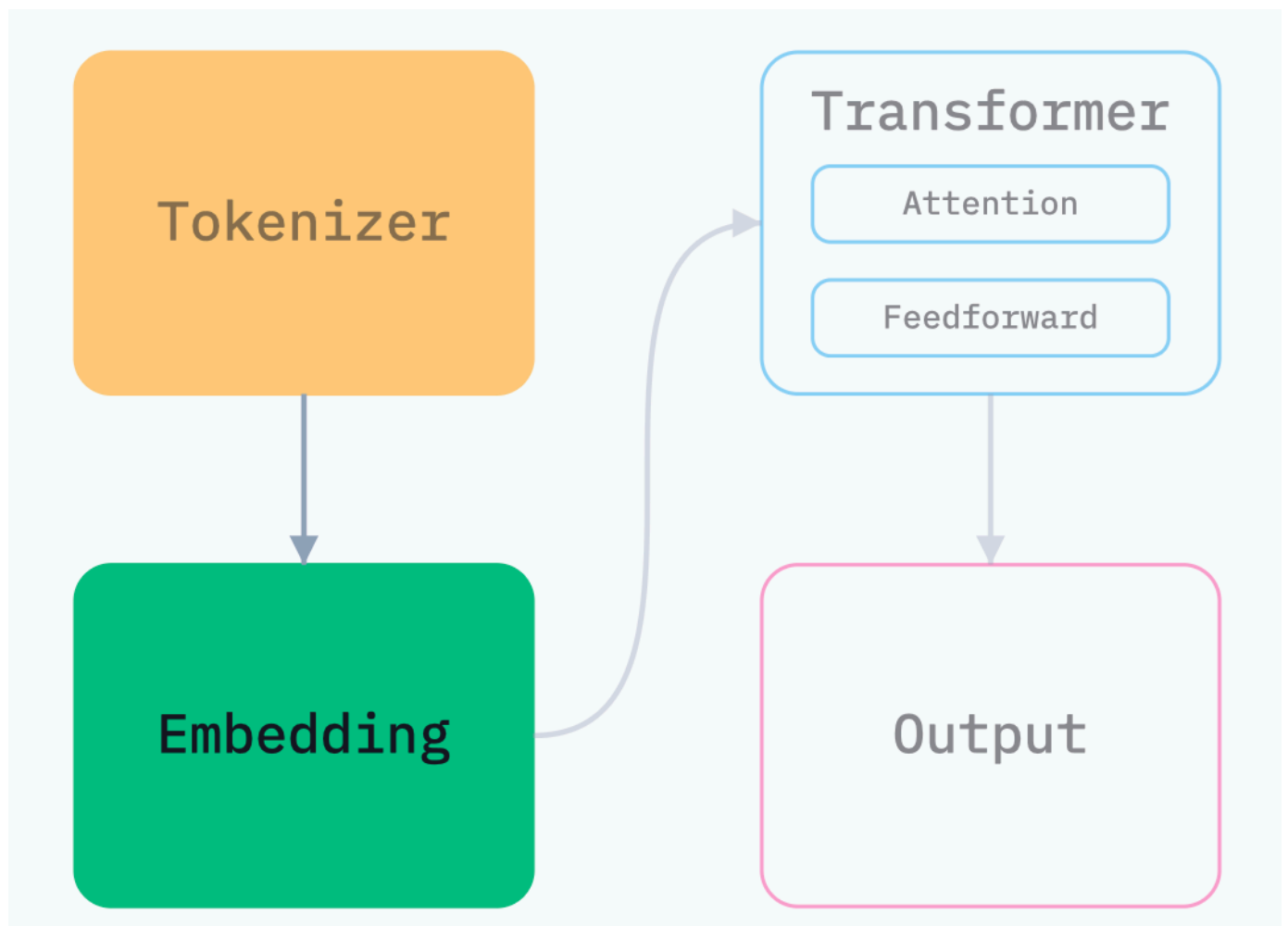
Inference должен когда-то закончиться. У LLM есть разные «специальные» токены, которые она может выдавать, и один из них сигнализирует о завершении ответа. В токенизаторе GPT-5 это токен 199999. Это лишь один из способов завершения генерации. Вы также можете задать максимальное количество токенов через API, а у провайдеров могут быть и другие правила остановки, связанные с безопасностью.

Существуют и специальные токены для обозначения начала и конца сообщений в диалоге — именно так чат-модели вроде ChatGPT и Claude понимают, где заканчивается одно сообщение и начинается другое.

И последнее про токенизаторы: их много. Токенизатор ChatGPT отличается от токенизатора Claude. Даже разные модели OpenAI используют разные токенизаторы. У каждого свои правила разбиения текста на токены. Если хотите посмотреть, как разные токенизаторы режут текст, загляните в [tiktokenizer](#).

Теперь, когда мы разобрались с токенами, поговорим об embeddings.

Embedding



Токены, полученные от токенизатора, теперь поступают на стадию embedding. Чтобы понять, что такое embedding, полезно сначала разобраться, какова цель модели.

Когда люди решают задачу с помощью кода, они пишут функции, которые принимают входные данные и возвращают результат. Например, перевод температуры из Фаренгейтов в Цельсии.


```
// Copy code
function fahrenheitToCelsius(fahrenheit) {
    return ((fahrenheit - 32) * 5) / 9;
}
```

Мы можем подать в `fahrenheitToCelsius` любое число и получить правильный ответ. Но что если у нас задача, где формула неизвестна? Что если у нас есть только загадочная таблица входов и выходов?

Input	Output
21	73
2	3
10	29
206	1277

Я не ожидаю, что вы узнаете эту функцию, хотя замечу, что ChatGPT сразу же её определяет, если вставить скриншот в приложение.

Когда мы знаем правильный результат для каждого входа, но не знаем саму функцию, мы можем «обучить» модель восстановить эту функцию. Для этого мы даём модели заготовку — тот самый огромный граф математических операций — и изменяем его, пока модель не сойдётся к нужной функции. После каждого обновления графа мы прогоняем через него входные данные и смотрим, насколько близко он подошёл к правильным выходам. Мы повторяем это, пока результат нас не устроит. Это и есть обучение.

Оказывается, при обучении модели генерировать корректный текст очень важно уметь определять, насколько два предложения похожи. Но похожи в каком смысле? Они могут быть одинаково грустными, смешными или заставляющими задуматься. Похожими по длине, ритму, тону, языку, словарю, структуре. Существует огромное количество измерений, по которым можно сравнивать предложения, и они могут быть похожи по одним измерениям и различаться по другим.

У токенов нет измерений. Это просто целые числа. А вот `embeddings` — имеют. Имеют много измерений.

Embedding — это массив длины n , представляющий точку в n -мерном пространстве. Если n равно 3, `embedding` может выглядеть как `[10, 4, 2]`, то есть точка с координатами $x=10$, $y=4$, $z=2$ в трёхмерном пространстве. При обучении LLM каждому токенту сначала назначается случайное положение в этом пространстве, а затем процесс обучения

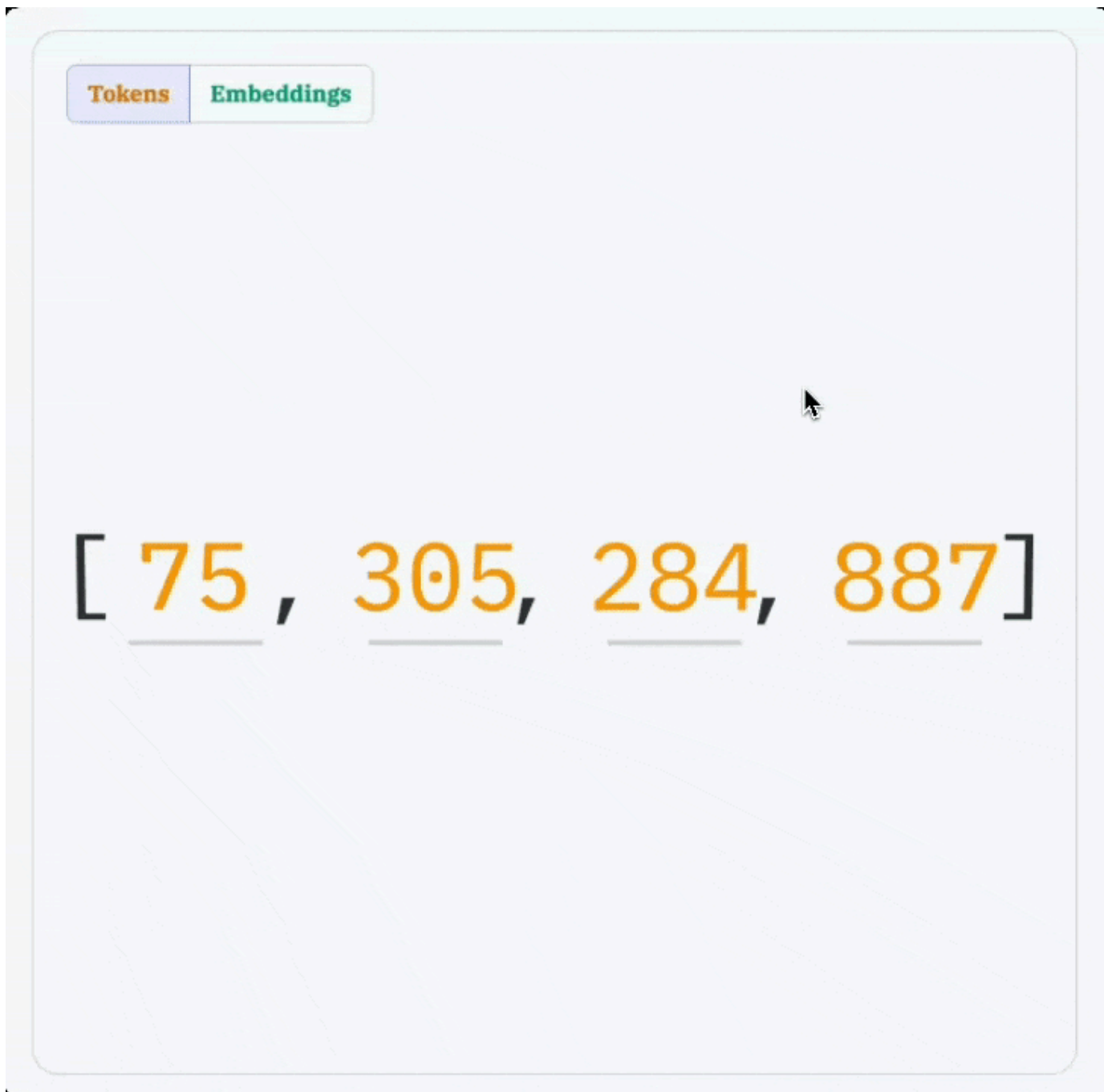
постепенно сдвигает все токены, пока не будет найдено такое расположение, которое даёт наилучшие результаты.

Стадия embedding начинается с того, что для каждого токена извлекается его embedding. В псевдокоде это может выглядеть так:

```
// Copy code
// Created during training, never changes during inference.
const EMBEDDINGS = [...];

function embed(tokens) {
  return tokens.map(token => {
    return EMBEDDINGS[token];
  });
}
```

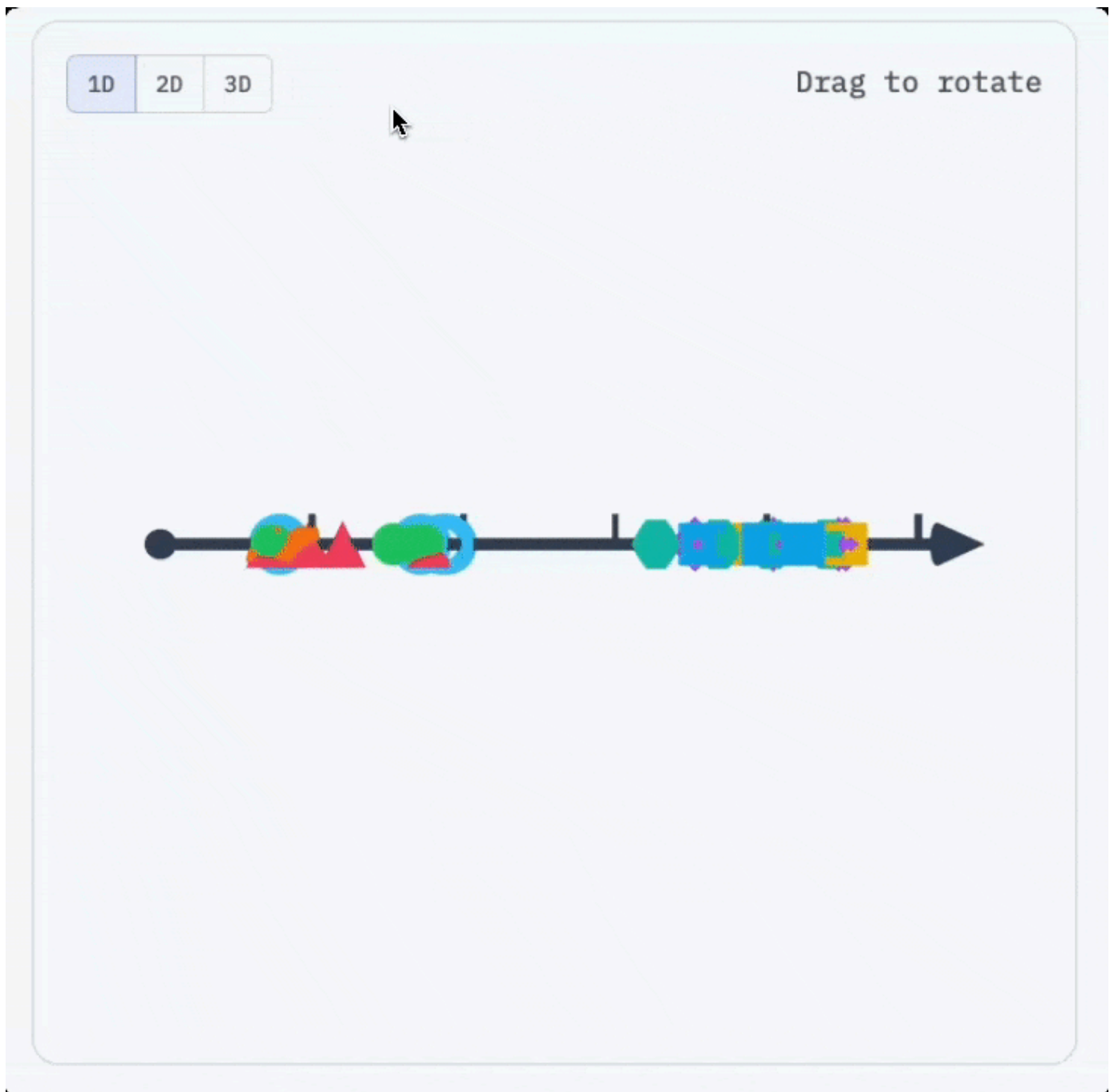
Мы берём токены — массив целых чисел — и превращаем их в массив embeddings. Массив массивов, или матрицу.



Токены [75, 305, 284, 887] преобразуются в матрицу трёхмерных embeddings.

Чем больше измерений у embeddings, тем по большему числу осей модель может сравнивать предложения. Мы говорили о embeddings с тремя измерениями, но современные модели используют embeddings с тысячами измерений. У самых больших моделей их больше 10 000.

Чтобы показать ценность дополнительных измерений, ниже я привёл восемь групп цветных фигур, которые сначала находятся в одномерном пространстве. Они лежат на линии и представляют собой хаотичную мешанину. Но по мере добавления измерений становится видно, что на самом деле это восемь отдельных, связанных между собой групп.



Три измерений — это максимум, который я могу показать визуалью, дальше вам придётся подключить воображение, чтобы представить, что можно сделать с тысячами измерений.

Есть ещё одна важная вещь, которую делает стадия `embedding`. После получения `embeddings` токена в них кодируется позиция токена внутри промпта. Я не углублялся в детали реализации — лишь настолько, чтобы понимать, что это почти не влияет на кэширование промптов, — но без этого LLM не смогла бы определить порядок токенов в тексте.

Обновим наш псевдокод, предполагая, что существует функция `encodePosition`. Она принимает `embeddings` и позицию и возвращает новые `embeddings` с закодированной позицией.

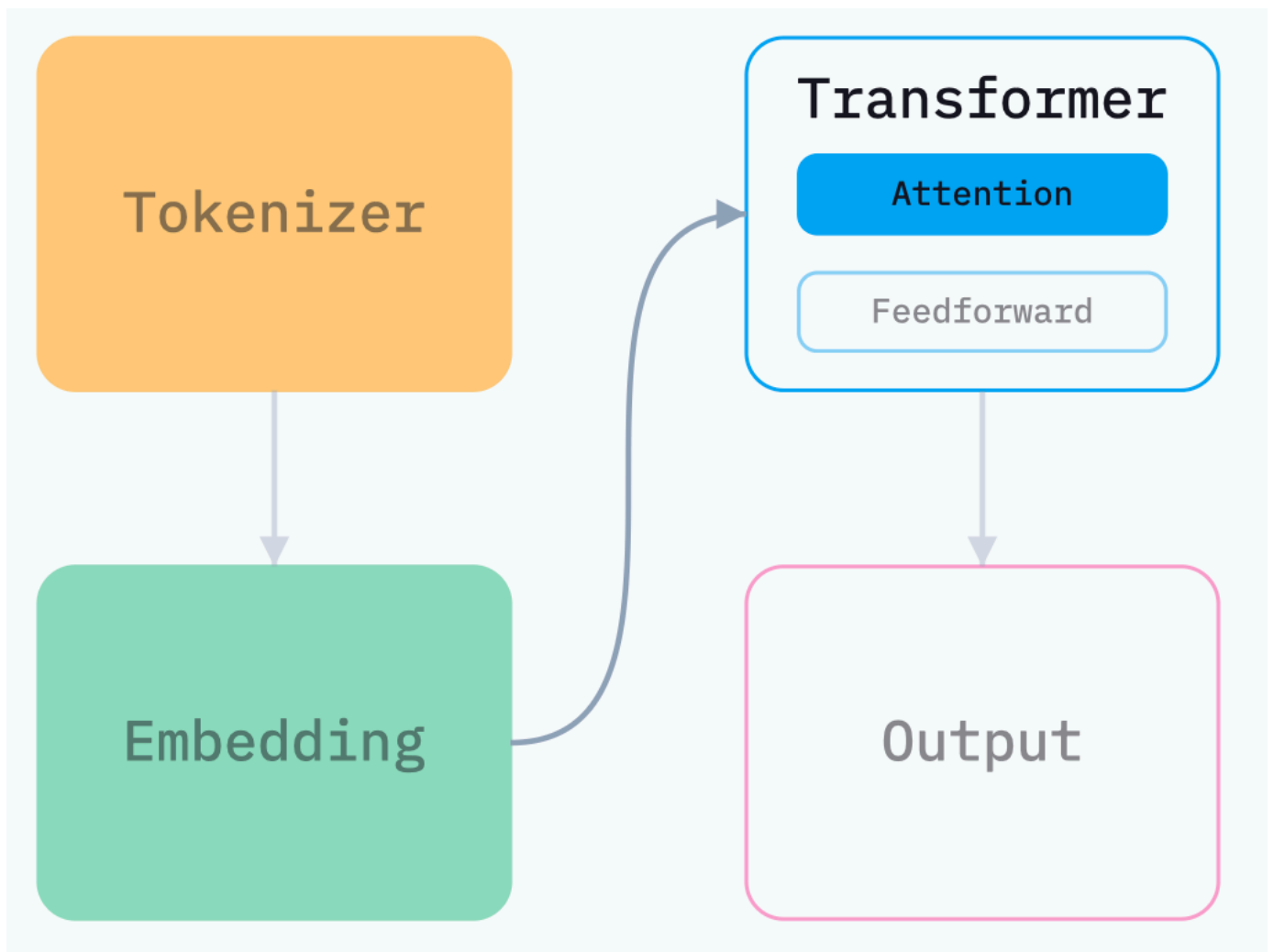
```
// Copy code
const EMBEDDINGS = [...];

// Input: array of tokens (integers)
function embed(tokens) {
  // Output: array of n-dimensional embedding arrays
  return tokens.map((token, i) => {
    const embeddings = EMBEDDINGS[token];
    return encodePosition(embeddings, i);
  });
}
```

Итого: embeddings — это точки в n-мерном пространстве, которые можно рассматривать как семантический смысл представляемого текста. Во время обучения каждый токен перемещается в этом пространстве так, чтобы находиться рядом с другими, похожими токенами. Чем больше измерений, тем более сложным и тонким может быть представление каждого токена у LLM.

Всё, что мы сделали на стадиях токенизации и embedding, было нужно, чтобы превратить текст в форму, с которой LLM может работать. Теперь давайте посмотрим, как выглядит эта работа на стадии трансформера.

Трансформер



Стадия трансформера — это про то, чтобы взять embeddings на вход и «переставить» их внутри n -мерного пространства. Делается это двумя способами, и мы сосредоточимся только на первом: attention. Про «Feedforward» и стадию вывода мы говорить не будем ().

Задача механизма attention — помочь LLM понять взаимосвязи между токенами в промпте, позволяя токенам влиять на положения друг друга в n -мерном пространстве. Для этого embeddings токенов промпта комбинируются взвешенным образом. На вход подаются embeddings всего промпта, а на выходе получается один новый embedding — взвешенная комбинация всех входных embeddings.

Например, если бы у нас был промпт «Mary had a little», и он превращался бы в 4 токена: Mary, had, a и little, механизм attention мог бы решить, что для генерации следующего токена нужно взять:

- 63% embeddings слова Mary
- 16% embeddings слова had
- 12% embeddings слова a
- 9% embeddings слова little

А затем он бы объединил их, масштабировав embeddings по весам и сложив. Так LLM «понимает», насколько ей стоит учитывать, то есть «внимательно смотреть» (attend) на каждый токен в промпте.

Это самая сложная и абстрактная часть процесса на данный момент. Сначала я покажу её в виде псевдокода, а затем посмотрим, как embeddings меняются при прохождении через неё. Я хотел сделать этот раздел менее «математичным», но тут трудно полностью избежать математики. Вы справитесь, я в вас верю.

Большинство вычислений в attention — это умножение матриц. Единственное, что вам нужно знать об умножении матриц для понимания этой статьи: размер (shape) выходной матрицы определяется размерами входных. У результата всегда столько же строк, сколько у первой матрицы, и столько же столбцов, сколько у второй.

$$\begin{bmatrix} & \mathbf{2 \times 3} \\ 1.00 & 2.00 & 3.00 \\ 4.00 & 5.00 & 6.00 \end{bmatrix} \times \begin{bmatrix} \mathbf{3 \times 1} \\ 6.00 \\ 7.00 \\ 8.00 \end{bmatrix} = \begin{bmatrix} \mathbf{2 \times 1} \\ 44.00 \\ 107.00 \end{bmatrix}$$

Имея это в виду, вот как упрощённый механизм attention вычисляет веса, которые нужно назначить каждому токenu. В коде ниже я использую `*` как обозначение умножения матриц.

```
// Copy code
// Similar to EMBEDDINGS from the pseudocode
// earlier, WQ and WK are learned during
// training and do not change during inference.
//
// These are both n*n matrices, where n is the
// number of embedding dimensions. In our example
// above, n = 3.
const WQ = [...], [...], [...];
const WK = [...], [...], [...];

// The input embeddings look like this:
// [
//   [-0.1, 0.1, -0.3], // Mary
```

```
// [1.0, -0.5, -0.6], // had
// [0.0, 0.8, 0.6], // a
// [0.5, -0.7, 1.0] // little
// ]
function attentionWeights(embeddings) {
  const Q = embeddings * WQ;
  const K = embeddings * WK;
  const scores = Q * transpose(K);
  const masked = mask(scores);
  return softmax(masked);
}
```

Давайте посмотрим, как `embeddings` «протекают» через эту функцию.

Подождите, а что такое `WQ` и `WK`?

Помните, я говорил раньше, что `embeddings` для каждого токена сначала получают случайное положение, а затем в процессе обучения эти значения понемногу подкручиваются, пока модель не сойдётся к хорошему расположению?

`WQ` и `WK` устроены похоже. Это матрицы размера $n \times n$ (где n — размерность `embeddings`), и в начале обучения они тоже заполняются случайными значениями. Затем во время обучения их так же постепенно корректируют, чтобы помочь модели сойтись к хорошему решению.

Всё, что изменяется в процессе обучения, называется «параметром модели» (`model parameter`). Каждое число с плавающей запятой в векторах `embeddings` и внутри матриц `WQ` и `WK` — это один параметр. Когда вы слышите, что у модели «175 миллиардов параметров», речь идёт именно об этих числах.

А что именно представляют собой `WQ` и `WK` — мы, по сути, не знаем. По мере сходимости они начинают означать некоторое преобразование `embeddings`, которое помогает модели выдавать хороший результат. Они могут «делать что угодно», а интерпретация их содержимого — это открытая и активно развивающаяся область исследований.

Чтобы получить `Q` и `K`, мы берём `embeddings` и умножаем их на `WQ` и `WK` соответственно. У `WQ` и `WK` число строк и столбцов всегда равно числу измерений `embeddings` — в нашем случае это 3. Я выбрал здесь случайные значения для `WQ` и `WK`, а числа округлил до двух знаков после запятой для удобства чтения.

$$\begin{array}{c} \text{embeddings} \\ \begin{bmatrix} -0.06 & 0.13 & -0.30 \\ 0.95 & -0.49 & -0.64 \\ -0.05 & 0.75 & 0.65 \\ 0.51 & -0.72 & 0.98 \end{bmatrix} \end{array} \times \begin{array}{c} \text{WQ} \\ \begin{bmatrix} -0.74 & 0.91 & -0.21 \\ -0.51 & 0.43 & 0.18 \\ -0.56 & -0.21 & 0.76 \end{bmatrix} \end{array} = \begin{array}{c} \text{Q} \\ \begin{bmatrix} 0.15 & 0.07 & -0.19 \\ -0.10 & 0.79 & -0.78 \\ -0.71 & 0.15 & 0.64 \\ -0.56 & -0.06 & 0.51 \end{bmatrix} \end{array}$$

Итоговая матрица Q имеет 4 строки и 3 столбца. 4 строки — потому что у матрицы embeddings было 4 строки (по одной на токен), а 3 столбца — потому что у WQ было 3 столбца (по одному на измерение embeddings).

Вычисление K полностью такое же, только вместо WQ используется WK.

$$\begin{array}{c} \text{embeddings} \\ \begin{bmatrix} -0.06 & 0.13 & -0.30 \\ 0.95 & -0.49 & -0.64 \\ -0.05 & 0.75 & 0.65 \\ 0.51 & -0.72 & 0.98 \end{bmatrix} \end{array} \times \begin{array}{c} \text{WK} \\ \begin{bmatrix} 0.13 & -0.51 & -0.63 \\ -0.79 & 0.54 & -0.04 \\ 0.33 & -0.10 & -0.87 \end{bmatrix} \end{array} = \begin{array}{c} \text{K} \\ \begin{bmatrix} -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \end{bmatrix} \end{array}$$

Q и K — это «проекции» входных embeddings в новые n-мерные пространства. Это не исходные embeddings, но они вычислены на их основе.

Затем мы берём Q и K и перемножаем их. Мы берём транспонированную K — то есть «переворачиваем» её по диагонали — чтобы итоговая матрица стала квадратной: число строк и столбцов в ней равно числу токенов входного промпта.

$$\begin{array}{c} \text{Q} \\ \begin{bmatrix} 0.15 & 0.07 & -0.19 \\ -0.10 & 0.79 & -0.78 \\ -0.71 & 0.15 & 0.64 \\ -0.56 & -0.06 & 0.51 \end{bmatrix} \end{array} \times \begin{array}{c} \text{transpose(K)} \\ \begin{bmatrix} -0.21 & 0.30 & -0.39 & 0.96 \\ 0.13 & -0.68 & 0.36 & -0.74 \\ 0.29 & -0.03 & -0.56 & -1.15 \end{bmatrix} \end{array} = \begin{array}{c} \text{scores} \\ \begin{bmatrix} -0.08 & 0.00 & 0.08 & 0.31 \\ -0.10 & -0.54 & 0.76 & 0.21 \\ 0.36 & -0.33 & -0.04 & -1.53 \\ 0.26 & -0.15 & -0.09 & -1.08 \end{bmatrix} \end{array}$$

Эти scores — представление того, насколько важен каждый токен для генерации следующего токена. Число в левом верхнем углу, -0.08, — это «насколько важна Mary для had». Затем строкой ниже, -0.10, — это «насколько важна Mary для a». Я покажу это визуально после матричной части. Всё, что будет дальше, — это превращение scores в веса, которые можно использовать для смешивания embeddings.

Первая проблема этой матрицы scores в том, что она позволяет будущим токенам влиять на прошлое. В верхней строке мы знаем только «Mary», значит, именно она и должна быть единственным словом, которое влияет на генерацию «had». То же самое во второй строке: мы знаем «Mary» и «had», значит только они двое должны влиять на генерацию «а», и так далее.

Чтобы исправить это, мы применяем треугольную маску к матрице, чтобы «обнулить» будущие токены. Но вместо нулей мы ставим минус бесконечность. Почему — объясню чуть позже.

$$\text{mask} \left(\begin{array}{c|c} & \text{scores} \\ \hline & \begin{bmatrix} -0.08 & 0.00 & 0.08 & 0.31 \\ -0.10 & -0.54 & 0.76 & 0.21 \\ 0.36 & -0.33 & -0.04 & -1.53 \\ 0.26 & -0.15 & -0.09 & -1.08 \end{bmatrix} \end{array} \right) = \begin{array}{c|c} & \text{masked} \\ \hline & \begin{bmatrix} -0.08 & -\infty & -\infty & -\infty \\ -0.10 & -0.54 & -\infty & -\infty \\ 0.36 & -0.33 & -0.04 & -\infty \\ 0.26 & -0.15 & -0.09 & -1.08 \end{bmatrix} \end{array}$$

Вторая проблема в том, что эти scores — произвольные числа. Нам было бы намного удобнее, если бы это было распределение, которое по каждой строке суммируется в 1. Именно это делает функция softmax. Детали работы softmax тут не важны: она немного сложнее, чем «разделить каждое число на сумму строки», но результат тот же — сумма по каждой строке равна 1, и каждое число находится между 0 и 1.

$$\text{softmax} \left(\begin{array}{c|c} & \text{masked} \\ \hline & \begin{bmatrix} -0.08 & -\infty & -\infty & -\infty \\ -0.10 & -0.54 & -\infty & -\infty \\ 0.36 & -0.33 & -0.04 & -\infty \\ 0.26 & -0.15 & -0.09 & -1.08 \end{bmatrix} \end{array} \right) = \begin{array}{c|c} & \text{weights} \\ \hline & \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.61 & 0.39 & 0.00 & 0.00 \\ 0.46 & 0.23 & 0.31 & 0.00 \\ 0.38 & 0.25 & 0.27 & 0.10 \end{bmatrix} \end{array}$$

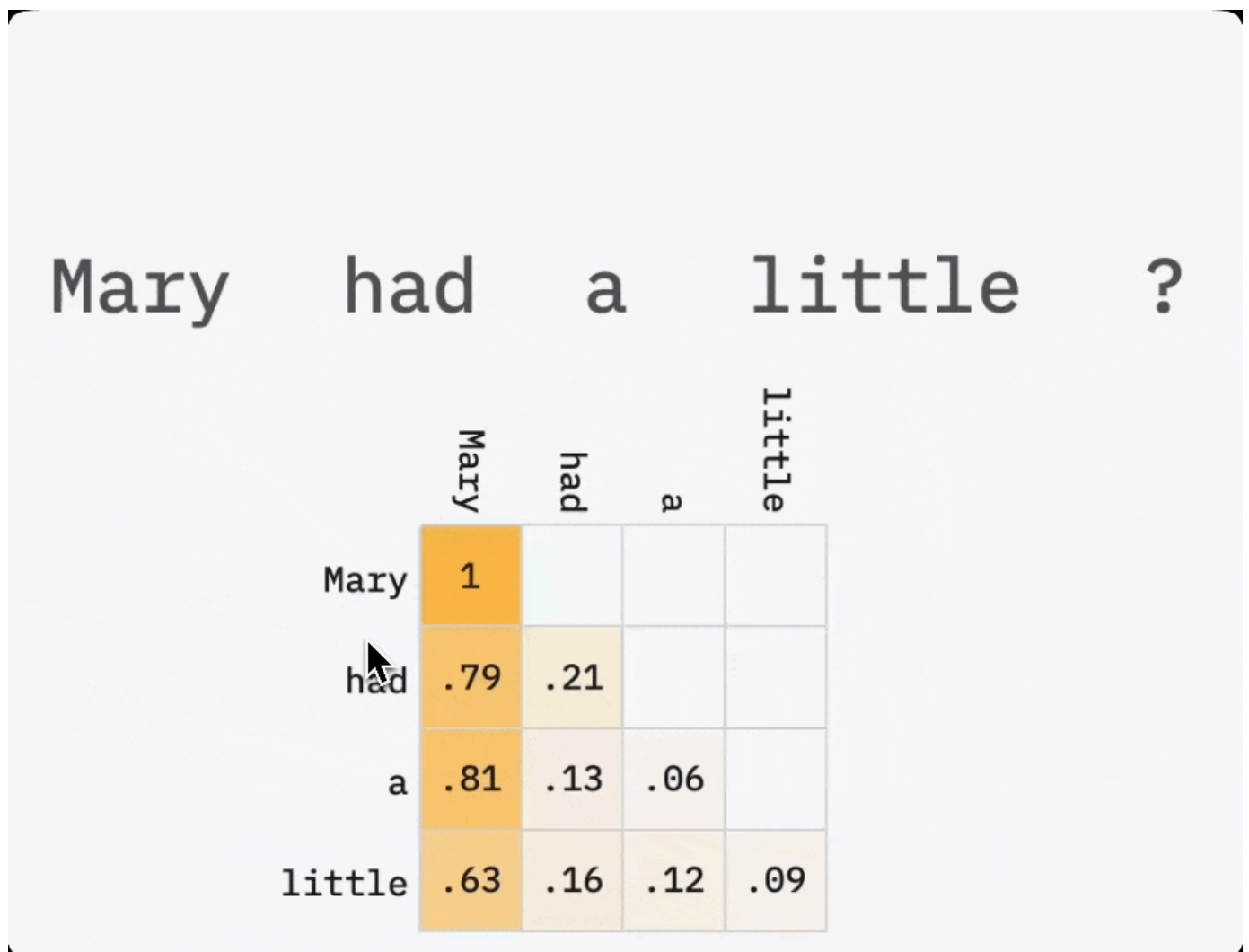
Чтобы пояснить, зачем здесь минус бесконечности, вот реализация softmax в коде:

```
// Copy code
function softmax(matrix) {
  return matrix.map(row => {
    const exps = row.map(x => Math.exp(x));
    const sumExps = exps.reduce((a, b) => a + b, 0);
    return exps.map(exp => exp / sumExps);
  });
}
```

```
});  
}
```

Это не совсем «сложить числа и поделить каждое на сумму». Сначала берётся `Math.exp` от каждого числа, то есть вычисляется e^x . Если бы вместо $-\infty$ мы подставляли нули, то `Math.exp(0) === 1`, и эти нули всё равно давали бы вес. А `Math.exp(-Infinity)` равно 0 — именно то, что нам нужно.

Сетка ниже показывает пример attention-весов для промпта «Mary had a little». Эти веса не совпадают с вычислениями выше, потому что я взял их из версии GPT-2, которая работает на отличном сайте Transformer Explained. То есть это реальные веса реальной, хоть и старой, модели.



В первой строке у нас есть только «Mary», поэтому Mary даёт 100% вклада в «had». Во второй строке Mary даёт 79%, а «had» — 21% при генерации «а», и так далее. Вероятно, неудивительно, что слово, которое LLM считает самым важным в этом предложении, — это «Mary»: у неё максимальный вес в каждой строке. Если бы я попросил вас продолжить фразу «Jessica had a little», вряд ли вы выбрали бы «lamb».

Осталось только смешать embeddings токенов — к счастью, это заметно проще, чем вычислять веса.

```
// Copy code
// Learned during training, doesn't change
// during inference. This is also an n*n matrix,
// where n is the number of embedding dimensions.
const WV = [...], [...], ...];

function attention(embeddings) {
  const V = embeddings * WV;
  // This is the `attentionWeights` function from
  // the section above. We're wrapping it in
  // this `attention` function.
  const weights = attentionWeights(embeddings);
  return weights * V;
}
```

Как и раньше, у нас есть матрица WV, заданная на этапе обучения. С её помощью мы получаем матрицу V из embeddings токенов.

$$\begin{bmatrix} \text{embeddings} \\ -0.06 & 0.13 & -0.30 \\ 0.95 & -0.49 & -0.64 \\ -0.05 & 0.75 & 0.65 \\ 0.51 & -0.72 & 0.98 \end{bmatrix} \times \begin{bmatrix} \text{WV} \\ 0.96 & 0.62 & -0.40 \\ -0.90 & 0.06 & -0.55 \\ -0.17 & -0.09 & 0.59 \end{bmatrix} = \begin{bmatrix} \text{V} \\ -0.12 & -0.00 & -0.23 \\ 1.46 & 0.62 & -0.49 \\ -0.83 & -0.04 & -0.01 \\ 0.97 & 0.18 & 0.78 \end{bmatrix}$$

Почему бы не смешивать embeddings напрямую?

Когда мы получаем Q и K, а затем перемножаем их, чтобы вычислить attention-веса, мы работаем исключительно с тем, насколько токены релевантны друг другу. Embeddings кодируют массу семантической информации о токенах: одно измерение может означать «цвет», другое — «размер», третье — «грубость» и так далее. Веса используют сходство, чтобы определить релевантность.

WV позволяет модели дополнительно решить, какие измерения стоит протащить дальше. В предложении «Mary had a little» важна Mary как имя. При этом модель могла выучить много про коктейль Bloody Mary или про Mary, Queen of Scots. Для детской считалки это не имеет отношения и лишь добавит шума, если тащить это дальше.

Поэтому WV позволяет отфильтровать нерелевантные признаки перед тем, как смешивать embeddings.

Дальше мы перемножаем V на веса, которые вычислили, и на выходе получаем новый набор embeddings:

$$\begin{bmatrix} \text{weights} \\ 1.00 & 0.00 & 0.00 & 0.00 \\ 0.61 & 0.39 & 0.00 & 0.00 \\ 0.46 & 0.23 & 0.31 & 0.00 \\ 0.38 & 0.25 & 0.27 & 0.10 \end{bmatrix} \times \begin{bmatrix} V \\ -0.12 & -0.00 & -0.23 \\ 1.46 & 0.62 & -0.49 \\ -0.83 & -0.04 & -0.01 \\ 0.97 & 0.18 & 0.78 \end{bmatrix} = \begin{bmatrix} \text{output} \\ -0.12 & -0.00 & -0.23 \\ 0.50 & 0.24 & -0.33 \\ 0.02 & 0.13 & -0.22 \\ 0.20 & 0.16 & -0.14 \end{bmatrix}$$

Итоговый результат механизма attention — последняя строка этой матрицы output. Вся контекстная информация из предыдущих токенов «вмешалась» в эту последнюю строку через attention, но чтобы получить её, пришлось вычислить и все предыдущие строки.

В итоге: embeddings заходят на вход, а на выходе получается новый embedding. Механизм attention проделал много тонкой математики, чтобы смешать токены в пропорции их важности, опираясь на матрицы WQ , WK и WV , которые он выучил во время обучения. Именно этот механизм позволяет LLM понимать, что важно в её контекстном окне, и почему.

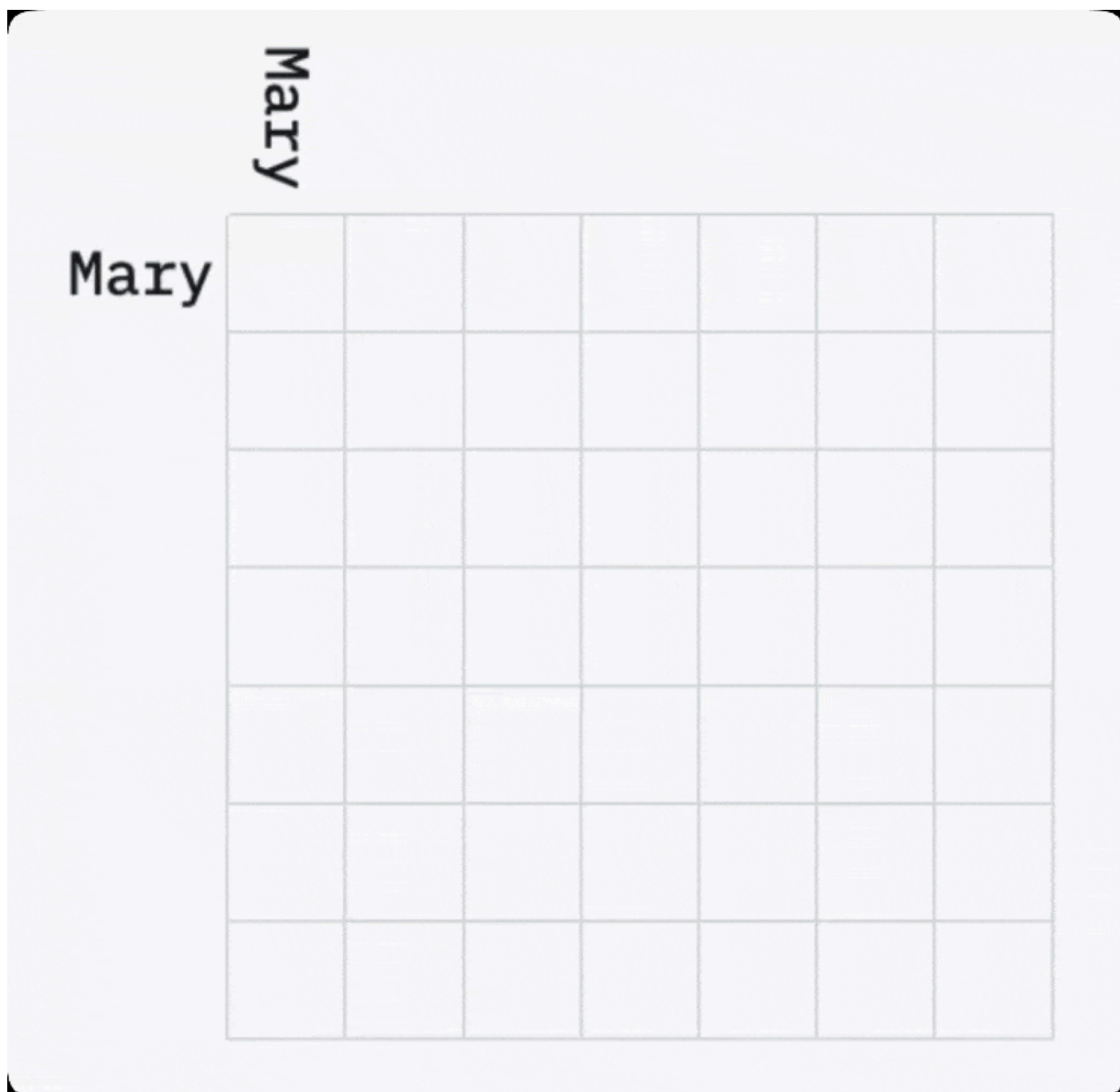
И вот теперь мы наконец знаем всё, что нужно, чтобы говорить о кэшировании.

В attention есть не только это

То, что я показал, — упрощённая (да, серьёзно) версия attention, которая нужна, чтобы подчеркнуть самое важное для кэширования промптов. На практике там есть дополнительные детали, и если хочется копнуть глубже, рекомендую видео [3blue1brown про attention](#).

Кэширование промптов

Давайте снова посмотрим на сетку выше — но теперь увидим, как она заполняется по мере генерации каждого нового токена в цикле inference.



Каждый новый токен добавляется ко входу и весь ввод заново перерабатывается целиком. Но приглядитесь: ни один из предыдущих весов не меняется. Вторая строка всегда 0.79 и 0.21. Третья строка всегда 0.81, 0.13, 0.06. Мы повторяем массу вычислений, которые повторять не нужно. Большинство умножений матриц для «Mary had a little» не требуется, если вы только что закончили обрабатывать «Mary had a» — а именно так и устроен цикл inference в LLM.

Избежать этих повторных вычислений можно, если сделать два изменения в цикле inference:

1. Кэшировать матрицы K и V на каждой итерации.
2. Передавать в модель только самый новый токен, а не весь промпт целиком.

Пройдёмся по матричным умножениям ещё раз, но теперь у нас заэкшированы K и V для первых 4 токенов, а на вход мы подаём embeddings только одного токена. Да, это снова математика, простите, но в основном всё то же самое, что выше, и мы пройдемся быстро.

Вычисление нового Q даёт на выходе всего одну строку. WQ не изменился по сравнению с прошлым разом.

$$\begin{bmatrix} \text{embeddings} \\ 0.20 & -0.10 & 0.70 \end{bmatrix} \times \begin{bmatrix} \text{WQ} \\ -0.74 & 0.91 & -0.21 \\ -0.51 & 0.43 & 0.18 \\ -0.56 & -0.21 & 0.76 \end{bmatrix} = \begin{bmatrix} \text{Q(new)} \\ -0.49 & -0.01 & 0.48 \end{bmatrix}$$

Затем вычисление нового K тоже даёт на выходе одну строку, и WK также тот же.

$$\begin{bmatrix} \text{embeddings} \\ 0.20 & -0.10 & 0.70 \end{bmatrix} \times \begin{bmatrix} \text{WK} \\ 0.13 & -0.51 & -0.63 \\ -0.79 & 0.54 & -0.04 \\ 0.33 & -0.10 & -0.87 \end{bmatrix} = \begin{bmatrix} \text{K(new)} \\ 0.34 & -0.23 & -0.74 \end{bmatrix}$$

Но дальше мы берём эту новую строку и дописываем её к 4 строкам K, которые были заэкшированы с предыдущей итерации:

$$\begin{bmatrix} \text{cached K} \\ -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \end{bmatrix} \text{ append } \begin{bmatrix} \text{K(new)} \\ 0.34 & -0.23 & -0.74 \end{bmatrix} = \begin{bmatrix} \text{K} \\ -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \\ 0.34 & -0.23 & -0.74 \end{bmatrix}$$

Итак, теперь у нас есть матрица K для всех токенов в промпте, но мы посчитали только её последнюю строку.

Дальше в том же духе получаем новые scores:

$$\begin{bmatrix} \text{Q(new)} \\ -0.49 & -0.01 & 0.48 \end{bmatrix} \times \begin{bmatrix} \text{transpose(K)} \\ -0.21 & 0.30 & -0.39 & 0.96 & 0.34 \\ 0.13 & -0.68 & 0.36 & -0.74 & -0.23 \\ 0.29 & -0.03 & -0.56 & -1.15 & -0.74 \end{bmatrix} = \begin{bmatrix} \text{scores(new)} \\ 0.24 & -0.16 & -0.08 & -1.01 & -0.52 \end{bmatrix}$$

И новые веса:

$$\text{softmax}\left(\begin{bmatrix} \text{scores(new)} \\ 0.24 & -0.16 & -0.08 & -1.01 & -0.52 \end{bmatrix}\right) = \begin{bmatrix} \text{weights(new)} \\ 0.32 & 0.21 & 0.23 & 0.09 & 0.15 \end{bmatrix}$$

Всё это время мы считаем только то, что действительно нужно. Старые значения вообще не пересчитываются. Далее получаем новую строку V:

$$\begin{bmatrix} \text{embeddings} \\ 0.20 & -0.10 & 0.70 \end{bmatrix} \times \begin{bmatrix} \text{WV} \\ -0.74 & 0.91 & -0.21 \\ -0.51 & 0.43 & 0.18 \\ -0.56 & -0.21 & 0.76 \end{bmatrix} = \begin{bmatrix} \text{V(new)} \\ -0.49 & -0.01 & 0.48 \end{bmatrix}$$

И дописываем её к V, который был в кэше:

$$\begin{bmatrix} \text{cached V} \\ -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \end{bmatrix} \text{ append } \begin{bmatrix} \text{V(new)} \\ -0.49 & -0.01 & 0.48 \end{bmatrix} = \begin{bmatrix} \text{V} \\ -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \\ -0.49 & -0.01 & 0.48 \end{bmatrix}$$

И наконец перемножаем новые веса с новым V, получая итоговые новые embeddings:

$$\begin{bmatrix} \text{weights(new)} \\ 0.32 & 0.21 & 0.23 & 0.09 & 0.15 \end{bmatrix} \times \begin{bmatrix} \text{V} \\ -0.21 & 0.13 & 0.29 \\ 0.30 & -0.68 & -0.03 \\ -0.39 & 0.36 & -0.56 \\ 0.96 & -0.74 & -1.15 \\ -0.49 & -0.01 & 0.48 \end{bmatrix} = \begin{bmatrix} \text{embeddings(new)} \\ -0.08 & -0.09 & -0.08 \end{bmatrix}$$

Эта единственная новая строка embeddings — всё, что нам было нужно. Вся контекстная информация из предыдущих токенов уже «запечена» в неё благодаря закэшированным K и V.

Данные, которые кэшируются, — это результат embeddings WK и embeddings WV, то есть K и V. Поэтому кэширование промптов обычно называют «KV-кэшированием» (KV caching).

Cache

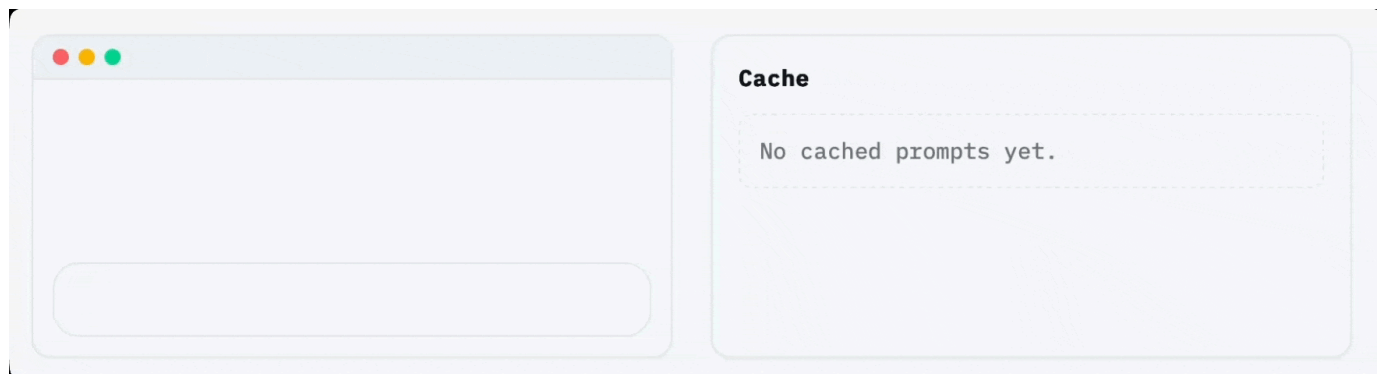
K			V		
-0.21	0.13	0.29	-0.21	0.13	0.29
0.30	-0.68	-0.03	0.30	-0.68	-0.03
-0.39	0.36	-0.56	-0.39	0.36	-0.56
0.96	-0.74	-1.15	0.96	-0.74	-1.15
0.34	-0.23	-0.74	-0.49	-0.01	0.48

Вот и всё: эти матрицы K и V выше — и есть те самые нули и единицы, которые провайдеры сохраняют в своих гигантских дата-центрах, чтобы давать нам токены в 10 раз дешевле и заметно ускорять ответы.

Провайдеры держат эти матрицы для каждого промпта 5–10 минут после запроса. Если вы отправляете новый запрос, который начинается с того же промпта, они переиспользуют закэшированные K и V вместо того, чтобы пересчитывать их заново. Самое классное —

можно частично совпасть с записью в кэше и всё равно использовать совпавшую часть, а не только «полное совпадение целиком».

Визуализация ниже по кругу перебирает несколько промптов с похожими префиксами, чтобы показать, как могут использоваться записи кэша. Время от времени кэш очищается, чтобы было видно, как он снова заполняется.



OpenAI и Anthropic делают кэширование очень по-разному. OpenAI делает всё автоматически, пытаясь по возможности направлять запросы на закэшированные записи. В моих экспериментах, если отправить запрос и сразу же отправить его повторно, мне удавалось получить hit rate около 50%. Учитывая, насколько большим может быть время до первого байта при длинных контекстных окнах, это может приводить к непредсказуемой производительности.

Anthropic дают больше контроля: вы сами решаете, когда кэшировать и на какой срок. За эту возможность вы платите, но в моих экспериментах Anthropic направляли на закэшированные записи в 100% случаев, когда я явно просил кэшировать промпт. Это может сделать их более подходящим выбором для приложений, где вы работаете с длинными контекстными окнами и вам нужна предсказуемая задержка.

Постойте, а как же temperature?

Провайдеры LLM предлагают ряд параметров, чтобы управлять «случайностью» того, что модель будет генерировать. Часто встречаются temperature, top_p и top_k. Эти параметры влияют на финальный шаг цикла inference, где модель выбирает токен на основе вероятностей, которые она назначила каждому токenu в своём словаре. Это происходит после того, как механизм attention сформировал финальные embeddings, поэтому кэширование промптов от этих параметров не зависит. Их можно свободно менять, не опасаясь «сломать» кэш промптов.

Русскоязычное сообщество про AI в разработке